IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 1 December 2025, accepted 30 December 2025, date of publication 5 January 2026, date of current version 8 January 2026.

*Digital Object Identifier 10.1109/ACCESS.2025.3650576*

RESEARCH ARTICLE

# Concept Drift Analysis Based on Isolation Forest for Effectively Detecting Network Attack in IoT Scenarios

**RENJIE CHU[1,2], MENGZHEN LUO[1], LEI YANG[3], JIANMING XIAO[1], AND YUANYUAN LIAO[1]**

[1]School of Information Engineering, Guilin University, Guilin 541006, China
[2]School of Data Sciences, Zhejiang University of Finance and Economics, Hangzhou 310018, China
[3]School of Finance and Law, Guilin University, Guilin 541006, China

Corresponding authors: Mengzhen Luo (ai_yuanm@163.com) and Renjie Chu (churenjie07@163.com)

This work was supported in part by the Enhancing Young and Middle-Aged Teacher's Research Basis Ability in Colleges of Guangxi under Grant 2024KY1679 and Grant 2017KY1327.

**ABSTRACT** In the Internet of Things era, massive network data streams are continuously generated, often exhibiting unpredictable distributional evolution. This dynamic nature poses significant challenges for effective analysis. To address this, we propose a novel double-layer concept drift detection algorithm based on isolation forest, termed IF-DLDD, designed to identify multiple types of concept drift. The framework consists of a detection layer, where isolation forest detects anomalies across consecutive sliding windows, and a verification layer, where a T-test is employed to precisely localize drift points and reduce false alarms. We evaluate IF-DLDD on both noisy synthetic datasets and the real-world CIC-IDS2017 intrusion detection dataset. The results demonstrate that IF-DLDD achieves timely and accurate detection across diverse drift types, maintaining a high detection rate with low false positives and strong noise robustness. In network intrusion detection tasks, IF-DLDD effectively identifies all attack drifts with minimal false positives and detection delays under practical parameter settings. These findings highlight IF-DLDD as a promising and practical approach for concept drift detection in dynamic data streams.

**INDEX TERMS** Data streams, isolation forest, concept drift, network intrusion detection.

## I. INTRODUCTION

The Internet of Things (IoT) has emerged as a global network that interconnects devices and communication systems, enabling seamless cooperation among heterogeneous entities and greatly facilitating daily life. However, the expansion of IoT also exposes cyberspace to a wide range of security risks and threats that must be carefully addressed [1]. Ensuring robust security is therefore imperative to support the continued growth and positive impact of IoT technologies.

Intrusion Detection Systems (IDS) are widely recognized as an effective approach to identifying malicious activities [2], [3]. By monitoring network traffic or system states, IDSs are able to detect various forms of network attacks. Nevertheless, IoT data streams are characterized by massive volume, high velocity, and continuous flow, which makes it impractical to process them in their entirety. As a result, batching or incremental processing strategies are commonly employed to handle such data streams efficiently [4].

A major challenge in analyzing data streams is the problem of concept drift [5], [6], [7], which arises when the underlying data distribution evolves over time, leading to degraded model performance. First introduced by Schlimmer and Granger in 1986 [5], the concept drift phenomenon has since been studied extensively and applied to diverse domains such as credit card fraud detection [8], [9], [10], network intrusion detection [11], [12], and healthcare systems [13], [14]. In this context, the term concept refers to the target distribution, while drift denotes a change in this distribution. Drift speed is typically defined as the reciprocal of the time required for a new concept to replace the old one [15], [16]. Four common types of drift have been identified:

The associate editor coordinating the review of this manuscript and approving it for publication was Bing Li.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

abrupt, gradual, incremental, and recurring [17], [18], [19]. Concept drift detection aims to identify change points or time intervals where such distributional shifts occur [6], [17], [20]. Importantly, in network intrusion detection, natural changes in normal traffic behavior may resemble attacks, highlighting the significance of concept drift analysis in IoT data streams.

A number of methods have been proposed for concept drift detection. For example, Gama [21] introduced the Drift Detection Method (DDM), which monitors the mean and standard deviation of classification error rates and defines warning and drift levels based on their variation. Similarly, the STEPD algorithm [22] compares the error rate of a local sliding window with that of the entire data stream, applying a chi-square test to determine whether significant differences exist. Although effective in certain scenarios, these approaches rely heavily on labeled data. Labeling at scale is costly and often infeasible, and the reliance on supervision makes such methods less generalizable. In addition, many detection techniques are highly sensitive to noise, leading to false alarms and reduced performance.

Other approaches focus on detecting drift via changes in data distribution. For instance, the Incremental Least-Square Density Difference (LSDD) method [23] measures the divergence between probability density functions to determine distributional shifts. While computationally efficient and effective for abrupt drifts, its performance degrades under gradual or recurring drift scenarios.

In general, the choice of detection statistics is crucial to the accuracy of drift detection, yet most existing methods still require fully labeled data. For example, DDM relies on sample mean and variance [21], while the Dynamic Residual Projection Method (DRPM) [24] combines anomaly detection with drift detection by comparing anomalous quantities between sliding windows. Although DRPM eliminates the need for labels, it remains sensitive to parameter settings and may suffer from instability in noisy environments.

To overcome these limitations, this paper proposes a novel Double-Layer Drift Detection framework based on Isolation Forest (IF-DLDD). The framework consists of two layers: a detection layer that leverages the isolation forest algorithm to capture distributional anomalies between consecutive windows, and a verification layer that employs a statistical t-test on anomaly scores to validate detected drifts and reduce false positives. By transforming drift detection into the assessment of window-level anomaly degrees, IF-DLDD enhances robustness while remaining label-free. The main contributions of this paper are summarized below:

1) The proposed method is fully unsupervised and does not require labeled data, making it practical for real-world applications where manual annotation is costly or infeasible.
2) By leveraging isolation forest, concept drift detection is reformulated as the evaluation of sliding-window anomaly scores, enabling efficient and noise-resilient detection.
3) The double-layer structure achieves high accuracy across multiple types of drift while maintaining a low false-positive rate.

## II. BACKGROUND

According to the types of statistics employed, existing concept drift detection methods can generally be divided into three main categories: error rate-based methods [21], [22], [25], data distribution-based methods [23], [26], and multiple hypothesis testing-based methods [27], [28], [29], [30].

Many drift detection algorithms in literature are derived from or combined with DDM [21], such as EDDM [31] and FW-DDM [32], which adopt statistical variations but share the same core principle of monitoring changes in classifier error rates to identify drift. Distribution-based approaches instead measure changes in data distributions, often through distance metrics between samples. For instance, the authors of [26] employed the Kullback–Leibler (K-L) divergence from information theory to detect drifts by quantifying changes in data distribution. Recently, deep learning approaches have also been explored in this context. For example, [33] proposed an unsupervised method leveraging deep learning representations to detect concept drift in real-time, demonstrating the capability of neural networks to capture complex distributional shifts.

Multiple hypothesis testing methods build upon these two categories but adopt more sophisticated frameworks, typically requiring higher computational cost. Alippi et al. [27] introduced a hierarchical architecture (HCDTs) with a detection layer and a verification layer. The detection layer quickly identifies potential distribution changes, while the verification layer re-examines them to reduce false positives. Two strategies were proposed: (1) estimating probability density functions via maximum likelihood and comparing them, and (2) applying classical hypothesis tests such as the Kolmogorov–Smirnov test. Similarly, the Multiscale Drift Detection Test (MDDT) [30] employed a t-test across two different scales, thereby reducing detection time and false positives. Other works, such as [29], proposed hierarchical tests that minimize label usage, while [34] developed a layered approach combining LFR [35] in the first stage with confusion-matrix-based validation in the second. Zhang et al. [28] further extended this idea into a three-layer framework that simultaneously examines both feature and label distributions, achieving improved accuracy and noise resistance in scenarios with frequent drift, such as text streams. Overall, multilayer detection structures demonstrate significant advantages in balancing false positives and detection delays. Such hierarchical strategies are particularly effective in mobile security scenarios. Notably, [36] designed a hybrid multilevel detection framework to identify malware on mobile devices, verifying that multi-stage detection remains robust even under concept drift.

In addition to these methods, recent studies have explored anomaly-detection-based approaches. For example, [37] presented an online, unsupervised anomaly detection

algorithm effective for both anomaly and drift detection, though it requires careful parameter tuning for different data streams. MADneSs [38] integrated multiple anomaly detection strategies to provide alerts based on unexpected behavioral trends in complex dynamic systems. Addressing the constraints of IoT environments, [39] introduced an online ensemble learning approach for anomaly detection, ensuring efficient adaptation to dynamic data streams. Similarly, the authors of [40] applied isolation forest to streaming anomaly detection, noting that drift alters the underlying distribution of anomalies. Isolation forest is particularly appealing for this task because it is unsupervised, computationally efficient, and non-parametric.

Building upon these insights, this paper proposes a double-layer concept drift detection framework. The isolation forest is well-suited for capturing anomalous behaviors in data streams and computing anomaly scores, while the double-layer architecture provides an additional verification stage to suppress false positives. The detailed design of the proposed framework is presented in the next section.

## III. THE PROPOSED APPROACH

To address the limitations of existing drift detection methods that often rely on labels and suffer from high false positive rates, we propose a double-layer concept drift detection algorithm based on isolation forest (IF-DLDD). This section first revisits the definition of concept drift and then introduces the proposed IF-DLDD framework, including its motivation, detection process, and theoretical foundation.

### A. PROBLEM STATEMENT

For concept drift analysis, we assume that during the time period $[0, t]$, a data stream $X = \{x_1, x_2, \ldots, x_n\}$ and its corresponding label set $Y = \{y_1, y_2, \ldots, y_n\}$, follows a certain distribution $F_{0,t}(X, Y)$, where $n$ denotes the number of observed instances in the data stream. If $F_{0,t}(X, y) \neq F_{t+1,\infty}(X, y)$, then it indicates that the data stream has concept drift at time $t + 1$.

Unlike many existing methods that depend on labels, our approach focuses on the distributional changes in anomaly scores. We assume that the data distribution characteristics change from a certain moment in a data stream. As the window slides, the corresponding anomaly amount should be significantly varied. According to the definition of concept drift, the probability distribution at time $t_1$ is $P_{t_1}(X, y)$, and after some time, the probability distribution of data stream changes to $P_{t_2}(X, y)$. If the distribution is significantly different, the data at $t_2$ is more likely to be abnormal. Thus, drift can be captured by monitoring the difference in the number of anomalies across sliding windows. In this work, we adopt isolation forest to compute anomaly scores and quantify these changes.

### B. DOUBLE-LAYER CONCEPT DRIFT DETECTION

The IF-DLDD framework employs a two-stage detection process consisting of a detection layer and a validation layer (Fig. 1). The detection layer uses isolation forest to compute anomaly scores within sliding windows and identify potential drift points with low computational overhead. The validation layer then applies a statistical test to verify whether the score distributions differ significantly, thereby reducing false positives.

As illustrated in Fig. 1, when the difference in anomaly quantities between the current and previous windows exceeds a threshold, the corresponding time point is marked as a potential drift. The validation layer is then activated to confirm whether the anomaly score distributions differ significantly using a t-test. This hierarchical design effectively filters out spurious detections, reducing false alarms and improving the accuracy of drift localization. The pseudocode of IF-DLDD is summarized in Algorithm 1.

**Algorithm 1** Double-Layer Concept Drift Detection

**Require:** $data\{X_i, y_i\}; n$
**Ensure:** concept drift time

```
for i = 1; i < n; i++ do
    Perform Detection layer
    if Detection layer detects potential drift point T then
        Perform Validation layer on T
        if Validation layer confirms the validity of T then
            save T
        else
            discard T, Restart Detection layer
        end if
    end if
end for
```

#### 1) DETECTION LAYER

The detection layer employs isolation forest [41], an unsupervised anomaly detection method particularly suitable for streaming data. Unlike supervised approaches, iForest does not require labeled training data and efficiently adapts to various types of anomalies and drifts. Its low computational complexity makes it well-suited for real-time applications.

Given a dataset $X = \{x_1, x_2, \ldots, x_n\}$, iForest includes two steps of training and testing to detect anomalies. During the training stage, iForest are established for the samples, forming iForests. Each tree undergoes testing, and the path length is recorded. Leveraging the rare and distinctive characteristics of anomalies, the iForest excels at isolating abnormal points, resulting in shorter corresponding path lengths. Subsequently, during the testing stage, iForest determines the abnormality of a sample based on its path length.

The anomaly score $s(x, n)$ for instance $x$ is defined as:

$$s(x, n) = 2^{-E(h(x))/c(n)}, \tag{1}$$

where $E(h(x))$ is the average of $h(x)$ from a collection of isolation trees. $c(n) = 2H(n-1) - (2(n-1)/n)$,

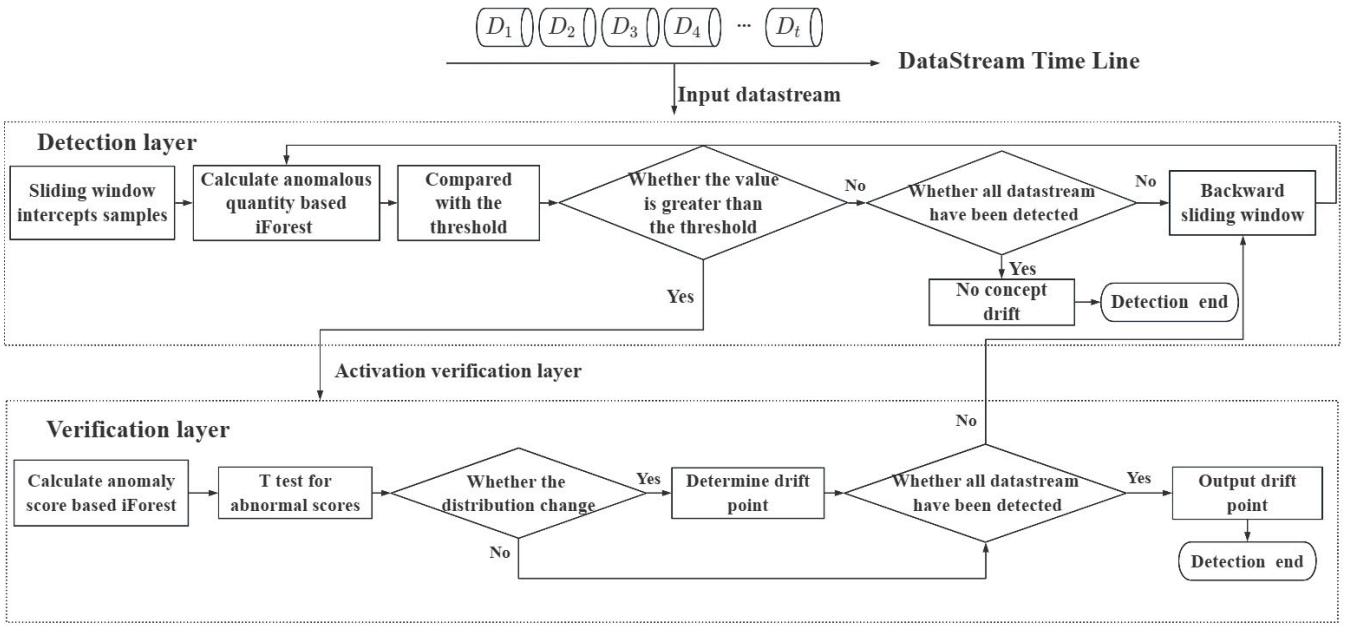



**FIGURE 1.** Double-layer detection framework of concept drift.

$H(i)$ is the harmonic number and it can be estimated by $ln(i)+ 0.5772156649$. According to formula (1), we have the following relations.

$$\begin{cases} E(h(x)) \to c(n), & s \to 0.5 \\ E(h(x)) \to 0, & s \to 1 \\ E(h(x)) \to n-1, & s \to 0 \end{cases}$$

We conclude that if $s$ is close to 1, the corresponding instances are definitely anomalies. Conversely, if $s$ is significantly smaller than 0.5, then they are safe to be regarded as normal. In contrast, if $s \approx 0.5$, the entire sample does not have any distinct anomaly.

As our study focuses on the detection of concept drift in data streams, the iForest able to adapt to changing patterns without requiring frequent model updates is a crucial advantage. This characteristic aligns well with the dynamic nature of our data.

In this paper, the anomaly count $D$ of a window $WS$ is defined as the number of instances whose isolation scores exceed a threshold $\tau$, as given in formula (2). In our experiments, we set $\tau = 0.6$, which effectively filters out borderline cases around 0.5 and retains only clearly abnormal samples. We further observed that varying $\tau$ within a small range([0.55, 0.65]) does not affect the qualitative behavior of IF-DLDD, indicating that the method is robust to the choice of $\tau$.

$$D = \sum_{x_i \in WS} \mathbf{1}\{s(x_i, n) > \tau\}. \tag{2}$$

If the anomaly count $D$ between the windows is significantly different, we consider that a concept drift has occurred. To precisely capture the actual changes in the anomaly count, we set a threshold, rather than merely observing fluctuations. This threshold setting helps reduce drift false positives, enabling us to more effectively identify real concept drift.

**Algorithm 2** Isolation Forest Anomaly Detection (Detection layer)

**Require:** $data\{X_i, y_i\}$, $n$, $step$, $\gamma$
**Ensure:** potential drift $T$

```
for k = 1 : s do
    WS1=data[(k − 1)step : (k − 1)step + γ, :]
    score1=s(WS1, n)
    Put the anomalous quantity into Dk;
    WS2=data[(kstep : kstep + γ, :]
    score2=s(WS2, n)
    Put the anomalous quantity into Dk;
    if Dk-Dk−1 > threshold then
        T[k-1][0] = k
        T[k-1][1] = (k-1)step+1
    else
        WS1 = [WS1 · WS2]
    end if
end for
```

Algorithm 2 shows the details for the detection layer. It compares the anomaly quantities $D$ in a fixed window $WS_1$ with that in a test window $WS_2$ using iForest. In this algorithm, $WS_1$ and $WS_2$ are allowed to overlap according to the window and step uncertainty settings. When the difference is larger than the threshold, we assume that there

**TABLE 1. Evaluation metrics.**

| | Real drift points | Real stationary points |
|---|---|---|
| **Detected drift points** | TP (True Positive) | FP (False Positive) |
| **Detected stationary points** | FN (False Negative) | TN (True Negative) |

is a potential drift point in $WS_2$. Specifically, $WS_1$ contains the initial $\gamma$ samples, we compute its $D_1$ and $score_1$. Then slide the window by step to get window $WS_2$ and calculate its $D_2$ and $score_2$. If $D_2 - D_1 > threshold$, the potential drift point is detected, then update $WS_1$ as $WS_1 = WS_2$. Otherwise, $[WS_1 \cdot WS_2]$ which means putting $WS_2$ into $WS_1$ and removing the duplicate samples. Repeat the above steps until all samples have been checked.

In our experiments, *threshold* is treated as a static parameter. For each dataset, we define a small set of candidate values for *threshold* and run IF-DLDD with these values to compare the resulting numbers of missed drifts, false alarms and detection delay. The value of *threshold* that provides the most satisfactory overall trade-off on that dataset is then selected and kept fixed for all subsequent experiments on that dataset.

### 2) VALIDATION LAYER

The detection layer, based on Isolation forest, provides an initial identification of candidate drift windows in the test stream. It can indicate that a drift is likely to have occurred within a window, but it cannot accurately determine the exact time of the change because each window contains a relatively large number of samples. In addition, random fluctuations in the anomaly scores may trigger spurious alarms. The validation layer is therefore introduced to refine these decisions by confirming or rejecting candidate drifts and by localizing the corresponding drift point within the detected window while reducing false positives.

Let a candidate drift window be denoted by $T = \{Z_1, Z_2, \ldots, Z_n\}$, where $Z_i$ is the anomaly score produced by Isolation forest for the $i$-th sample in the window. We split this window at an index $m$ into two consecutive subwindows,

$$T^{(1)} = \{Z_1, Z_2, \ldots, Z_m\}, \qquad T^{(2)} = \{Z_{m+1}, Z_{m+2}, \ldots, Z_n\}.$$

The anomaly scores in $T^{(1)}$ and $T^{(2)}$ are treated as two independent samples $X = \{X_1, \ldots, X_{n_1}\}$ and $Y = \{Y_1, \ldots, Y_{n_2}\}$, where $n_1 = m$ and $n_2 = n - m$.

To test whether a significant change in the mean anomaly score has occurred between the two subwindows, we adopt the two-sample t-test. Isolation forest anomaly scores are not strictly Gaussian, since they are bounded in $[0, 1]$. However, each score is obtained by averaging the path lengths over multiple random trees, and the validation layer further aggregates scores over a sliding window. In our experiments, the window size is at least 200 samples, so the aggregated scores within each subwindow can be regarded as approximately normal according to the central limit theorem. In such large-sample settings, the two-sample t-test is known to be robust to moderate deviations from normality. Let $\bar{X}$ and $\bar{Y}$ denote the sample means of $X$ and $Y$, and let $s_X^2$ and $s_Y^2$ be the corresponding sample variances. The test statistic is

$$t = \frac{\bar{X} - \bar{Y}}{\sqrt{s_X^2/n_1 + s_Y^2/n_2}}. \tag{3}$$

Under the null hypothesis $H_0 : \mu_X = \mu_Y$, the statistic in (3) approximately follows a t-distribution with degrees of freedom

$$\nu = \frac{\left(s_X^2/n_1 + s_Y^2/n_2\right)^2}{\frac{(s_X^2/n_1)^2}{n_1 - 1} + \frac{(s_Y^2/n_2)^2}{n_2 - 1}}. \tag{4}$$

For a given significance level $\alpha$, we reject $H_0$ if $t > t_\alpha(\nu)$, where $t_\alpha(\nu)$ is the upper-$\alpha$ quantile of the t-distribution with $\nu$ degrees of freedom.

In the IF-DLDD framework, a candidate window $T$ identified by the detection layer is processed by the validation layer as follows. First, $T$ is split into two subwindows $T^{(1)}$ and $T^{(2)}$ around the suspected change position. Then the Welch t-statistic (3) and the corresponding degrees of freedom (4) are computed using the anomaly scores in the two subwindows. If the statistic exceeds the critical value $t_\alpha(\nu)$, we conclude that the mean anomaly score in the post-drift segment $T^{(2)}$ is significantly larger than that in the pre-drift segment $T^{(1)}$, the candidate drift is confirmed, and the split index $m$ is recorded as the estimated drift point. Otherwise, the candidate drift is regarded as noise and the alarm is rejected. This procedure is summarized in Algorithm 3, where line 2 performs the window split, line 3 computes the t-statistic, and lines 4–7 update the concept boundaries only when the test indicates a statistically significant change.

## IV. EXPERIMENT AND RESULT ANALYSIS

To evaluate the effectiveness of IF-DLDD, we conduct experiments on seven artificial datasets and one network intrusion dataset, CIC-IDS2017. The evaluation metrics include the numbers of TP, FP, FN and the average detection delay. Their meanings are summarized in Table 1, and the delay is defined in (5).

The average delay represents the average delay between detected concept drift and actual concept drift. Its calculation formula is as follows:

$$\text{Delay} = \frac{\sum \text{delay at each drift}}{\text{total number of drift}}. \tag{5}$$

In order to comprehensively consider the above indicators and more accurately evaluate the results of concept drift detection, we redefine the statistic $F_1$ as follows:

$$F_1 = \frac{(1+\beta^2)\,\text{TP}}{(1+\beta^2)\,\text{TP} + \beta^2\,\text{FN} + \alpha\,\text{NormDelay} + \gamma\,\text{FP}}, \tag{6}$$

where NormDelay denotes the normalized delay given by

$$\text{NormDelay} = \frac{\text{Delay}}{\max(\text{Delay})}, \tag{7}$$

**Algorithm 3** T-Test Based Validation Layer

**Require:** potential drift window $T = \{Z_1, Z_2, \ldots, Z_n\}$ flagged at time index $k$, significance level $\alpha$, current concept set *concept*

**Ensure:** updated *concept* and drift decision (True/False)

choose a split index $m$ with $1 < m < n$
divide $T$ into two parts:
    $T^{(1)} = \{Z_1, Z_2, \ldots, Z_m\}$
    $T^{(2)} = \{Z_{m+1}, Z_{m+2}, \ldots, Z_n\}$
treat anomaly scores in $T^{(1)}$ and $T^{(2)}$ as two samples $X$ and $Y$
compute sample means $\bar{X}, \bar{Y}$ and sample variances $s_X^2, s_Y^2$
compute t-statistic $t$ according to (3)
compute degrees of freedom $\nu$ according to (4)
**if** $t > t_\alpha(\nu)$ **then**
    **Confirm** a concept drift at index $k$
    record $k$ as the drift point
    update concept boundaries in *concept*
    **return** True
**else**
    **Reject** this candidate drift (treat as noise)
    keep *concept* unchanged
    **return** False
**end if**

and max(Delay) is the maximum average delay among all compared methods on the same dataset.

This definition can be viewed as an extension of the traditional $F_1$ score. The numerator is identical to that of the standard $F_1$, while the denominator additionally penalizes large delays and many false positives through the weighted terms $\alpha \cdot \text{NormDelay}$ and $\gamma \cdot \text{FP}$. As a result, $F_1$ takes values in the interval (0, 1], and a larger value indicates better overall performance, in the sense of detecting as many drifts as possible, as early as possible, with as few spurious alarms as possible.

The parameters $\alpha$, $\beta$ and $\gamma$ control the relative importance of these three aspects. We fix $\beta = 1$ so that the contribution of TP and FN is consistent with the usual $F_1$ definition and detection accuracy remains the primary factor. The weight $\alpha$ governs the impact of detection delay: a larger $\alpha$ puts more emphasis on early detection and penalizes large delays more heavily, whereas a smaller $\alpha$ tolerates larger delays. The weight $\gamma$ regulates the influence of FP: larger $\gamma$ imposes a stronger penalty on false alarms, while smaller $\gamma$ allows more aggressive detectors that may generate more alarms.

In our experiments, we adopt $(\alpha, \beta, \gamma) = (0.6, 1, 0.3)$ as a reasonable compromise for our target applications, where missing or very late detections are considered more harmful than occasional false alarms. We also examined several nearby configurations, for example $\alpha \in \{0.4, 0.6, 0.8\}$ and $\gamma \in \{0.1, 0.3, 0.5\}$, and observed that the relative ranking of the compared methods does not change. This indicates that our conclusions are robust to the precise choice of these weights.

**TABLE 2.** Artificial dataset summary.

| Data | Instance | Feature | Class | Noise | Drift | Type |
|---|---|---|---|---|---|---|
| SEA | 60000 | 3 | 2 | 10% | 3 | Abrupt |
| UG-2C-5D | 200000 | 5 | 2 | none | 99 | Abrupt |
| Stagger | 100000 | 3 | 2 | none | 2 | Abrupt |
| Interchanging RBF | 200000 | 2 | 15 | none | 67 | Incremental |
| Hyperplane | 200000 | 10 | 2 | 5% | 39 | Gradual |
| Sine1 | 100000 | 2 | 2 | none | 4 | Abrupt, Re-occurring |
| Sine2 | 100000 | 2 | 2 | none | 4 | Abrupt, Re-occurring |

### A. ARTIFICIAL DATASET

To assess the generalization of IF-DLDD in concept drift detection for various categories, we consider the seven datasets in Table 2. It covers all the common types of concept drift.

The SEA dataset was proposed by Street and Kim in 2001 [42].It has 60000 samples with three features, and two classes. There are three drift points, in which concept drift occurs once every 15000 samples, which means four different concepts in total. Additionally, there is 10% noise in SEA to test the anti-noise ability.

The same as SEA, UG-2C-5D merely contains abrupt concept drift. It has two-class samples with five features and a total number of 200000. It drifts once every 2000 samples with 100 concepts and 99 drift points.

Stagger is a similar abrupt dataset with two classes. It has 100000 samples with three features. Finally yields three concepts and two drift points, which occur every 33333 data.

InterchangingRBF has 200000 samples with two features and a total number of 15 classes. Gaussian variables representing different classes substitute each other for every 3000 data until all gaussian sequences have changed. A total of 67 incremental drifts in this dataset.

The Hyperplane is a dataset generated by HyperplaneGenerator with a change of 0.001 probability. It contains two classes consisting of 200000 samples with ten features. Every 5000 data fluctuate once to generate 39 drift points covering 40 concepts. Furthermore, there is 5% noise.

Sine1 contains abrupt and reoccurring concepts. It is a two-class dataset consisting of 100000 samples with two features. The data obey a uniform distribution of [0, 1]. Its determination boundary is $y = sin(x)$, where concept drift occurs every 20000 samples and makes it a repeated concept. So there are five concepts and four drift points.

Compared to Sine1, the only difference of Sine2 is its determination boundary, $y = 0.5 + 0.3sin(3\pi x)$.

### B. RESULTS ON ARTIFICIAL DATASETS

The artificial datasets selected in this paper cover a variety of concept drift types. IF-DLDD contains two hyperparameters of window size and step, which are prominent in affecting

**TABLE 3.** Results on abrupt dataset using IF-DLDD.

| Data | Window | Step | TP | FP | FN | Delay | $F_1$ |
|---|---|---|---|---|---|---|---|
| SEA | **1000** | **600** | **3** | **0** | **0** | **2591** | **0.915** |
| | 1000 | 700 | 2 | 2 | 1 | 399 | 0.703 |
| | 1100 | 600 | 3 | 2 | 0 | 2799 | 0.833 |
| | 1200 | 700 | 2 | 0 | 1 | 2498 | 0.723 |
| | 1400 | 600 | 3 | 3 | 0 | 2392 | 0.809 |
| | 1500 | 500 | 2 | 1 | 1 | 999 | 0.725 |
| | 1700 | 600 | 3 | 1 | 0 | 2591 | 0.875 |
| UG-2C-5D | 200 | 130 | 56 | 18 | 43 | 920 | 0.696 |
| | 200 | 180 | 48 | 15 | 51 | 854 | 0.631 |
| | 300 | 120 | 57 | 14 | 42 | 870 | 0.709 |
| | **300** | **180** | **43** | **9** | **56** | **958** | **0.952** |
| | 400 | 130 | 44 | 13 | 55 | 818 | 0.597 |
| | 400 | 180 | 45 | 7 | 54 | 864 | 0.614 |
| Stagger | **1200** | **600** | **2** | **2** | **0** | **4577** | **0.817** |
| | 1300 | 600 | 2 | 3 | 0 | 7596 | 0.742 |
| | 2000 | 700 | 2 | 3 | 0 | 4331 | 0.772 |
| | 2500 | 620 | 2 | 2 | 0 | 9326 | 0.769 |
| | 2500 | 700 | 2 | 6 | 0 | 5704 | 0.648 |

**TABLE 4.** Results on incremental dataset using IF-DLDD.

| Data | Window | Step | TP | FP | FN | Delay | $F_1$ |
|---|---|---|---|---|---|---|---|
| InterchangingRBF | **300** | **150** | **48** | **23** | **19** | **949** | **0.785** |
| | 500 | 150 | 44 | 24 | 23 | 1322 | 0.741 |
| | 500 | 200 | 39 | 16 | 28 | 1435 | 0.700 |
| | 500 | 250 | 32 | 9 | 35 | 984 | 0.627 |
| | 700 | 150 | 46 | 22 | 21 | 1513 | 0.765 |
| | 700 | 200 | 38 | 17 | 29 | 1263 | 0.687 |

drift detection results. This section discusses the measured performance of IF-DLDD in different window sizes and step.

### 1) ABRUPT DATASET

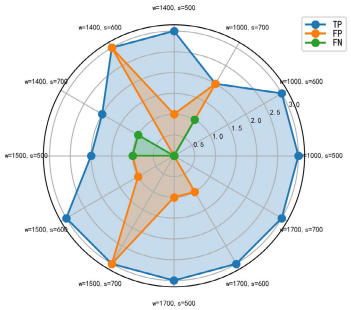
The abrupt datasets we consider are SEA, UG-2C-5D, and Stagger. Table 3 shows their results of FP, FN, Delay and $F_1$ with window size and step varied.

On the SEA dataset, IF-DLDD is capable of detecting all drift points correctly and free of FP and FN when window = 1000 and step = 600. However, the delay is relatively large, which is 2591, the corresponding $F_1$ score for this configuration attains its maximum, reaching 0.909. If we increase step to 700, the $F_1$ score decreases to 0.703, much smaller delay accompanied with 2 false alarms and 1 missing. Although the average delay is small, the results on UG-2C-5D show terrible performances with many missing reports. A reverse situation occurs on Stagger with heavy delay but yields zero false negative, which should be a concern.

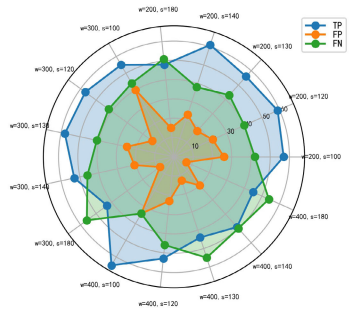
To intuitively observe the relationships between parameters and performances, we provide the radar images in Figure 2. From each subgraph, we can observe the influence of different windows and step sizes on the false positives and missed positives of the three abrupt datasets. For example, on the SEA and Stagger datasets, a small number of false positives exist in the detected drift points. Compared with UG-2C-5D, there are a lot of false negatives.

### 2) INCREMENTAL DATASET

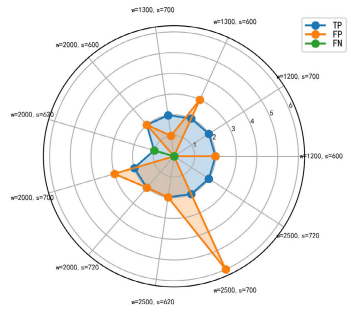
InterchangingRBF is taken as the experimental dataset of incremental concept drift and the experimental analysis of the influence of window and step size on false alarms missed




(a) SEA




(b) UG-2C-5D




(c) Stagger

**FIGURE 2.** The TP, FP and FN values of different windows and step over SEA, UG-2C-5D, Stagger datasets.

alarms. Delay is carried out on this dataset, as shown in Table 4:

As shown in Table 4 when window = 300, step = 150, IF-DLDD exhibits excellent detection performance. It detects 48 drift points, 70% drifts were detected. When window = 500, step = 250, IF-DLDD detects 32 drift points with 9 false alarms.

According to Figure 3, when window = 300 and step = 120, 53 drift points were accurately detected, the number of false alarms was 41, and the number of missed alarms was 14. In this case, the number of drift points detected is the largest, and the number of false alarms is also more. When the step is set to 140, and other parameters remain unchanged, 52 drift points can be accurately detected. Compared with step = 120, one correct drift point is reduced, but the number of false alarms FP is reduced to 30. When step = 150, most drift points can be detected under different

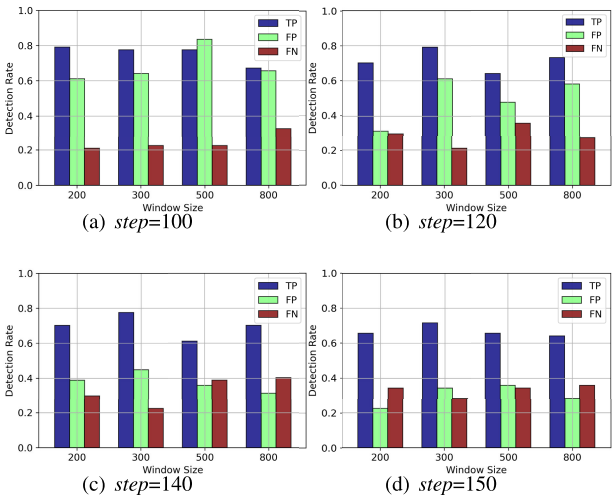



**FIGURE 3.** The TP, FP and FN values of the different windows over InterchangingRBF. The X-axis represents the window, and the Y-axis denotes the corresponding metric values.

**TABLE 5.** Results on gradual dataset using IF-DLDD.

| Data | Window | Step | TP | FP | FN | Delay | $F_1$ |
|---|---|---|---|---|---|---|---|
| Hyperplane | **300** | **250** | **31** | **9** | **8** | **1870** | **0.848** |
| | 500 | 300 | 27 | 9 | 12 | 1977 | 0.781 |
| | 600 | 300 | 29 | 5 | 10 | 2102 | 0.829 |
| | 700 | 450 | 28 | 4 | 11 | 2188 | 0.815 |
| | 1000 | 300 | 24 | 9 | 15 | 2049 | 0.726 |
| | 1100 | 600 | 17 | 3 | 22 | 2339 | 0.592 |
| | 1200 | 500 | 20 | 2 | 19 | 1699 | 0.667 |
| | 1300 | 600 | 21 | 3 | 18 | 2589 | 0.683 |
| | 1400 | 600 | 20 | 2 | 19 | 2648 | 0.665 |
| | 1500 | 700 | 16 | 0 | 23 | 2785 | 0.576 |

window sizes, and the number of false positives and missed positives is very small. When a step is set to 100, different window sizes can accurately detect nearly 80% drift points but with a high false positive rate at the same time.

### 3) GRADUAL DATASET

The hyperplane is taken as the experimental dataset of gradual concept drift, and the experimental analysis of the influence of window and step size on false positives, missed positives, and delay is conducted on this dataset, as shown in Table 5:

Several parameter settings can be correct and timely to detect most drift points in the Hyperplane dataset, and the false alarms are small. According to Table 5, when window = 300, step = 250, IF-DLDD detects 31 drift points, much smaller delay accompanied with 9 false alarms and 8 missing. Because there is noise in the data, it again shows that the algorithm in this paper has anti-noise ability.

### 4) MIXED DATASET

The mixed datasets we consider are Sine1 and Sine2. Table 6 shows their results of FP,FN, Delay and $F_1$ with window size and step varied.

On the Sine1 dataset, when window = 2900, step = 1400, the algorithm detects all the drift points without false positives or missing positives, but the delay reached nearly 9000. When window = 1200, step = 600, the algorithm also detects all drift points with false positives, and the delay was reduced to 1496. On the Sine2 dataset, when window = 2000 and 2800, step = 1000, the algorithm shows better performance in false alarm, missing alarm and delay respectively.

**TABLE 6.** Results on mixed dataset using IF-DLDD.

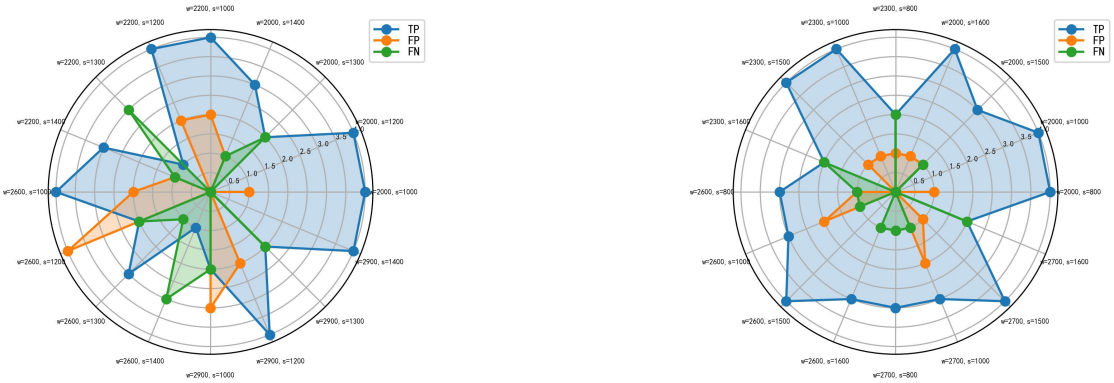
| Data | Window | Step | TP | FP | FN | Delay | $F_1$ |
|---|---|---|---|---|---|---|---|
| Sine1 | **800** | **700** | **4** | **2** | **0** | **3640** | **0.905** |
| | 1000 | 700 | 4 | 3 | 0 | 8029 | 0.848 |
| | 1200 | 600 | 4 | 3 | 0 | 1496 | 0.889 |
| | 2000 | 1300 | 2 | 0 | 2 | 7296 | 0.617 |
| | 2200 | 1000 | 4 | 2 | 0 | 5998 | 0.889 |
| | 2600 | 1000 | 4 | 2 | 0 | 7998 | 0.876 |
| | 2900 | 1000 | 2 | 3 | 2 | 4997 | 0.553 |
| | **2900** | **1400** | **4** | **0** | **0** | **8997** | **0.930** |
| Sine2 | 1300 | 500 | 4 | 4 | 0 | 4498 | 0.844 |
| | 1400 | 600 | 3 | 1 | 1 | 9730 | 0.759 |
| | **2000** | **1000** | **4** | **0** | **0** | **6998** | **0.949** |
| | **2300** | **1000** | **4** | **1** | **0** | **1999** | **0.950** |
| | 2400 | 1400 | 3 | 3 | 1 | 2799 | 0.743 |
| | 2500 | 1000 | 3 | 4 | 1 | 1332 | 0.724 |
| | **2600** | **1500** | **4** | **0** | **0** | **7748** | **0.944** |
| | 2700 | 1000 | 3 | 2 | 1 | 1332 | 0.781 |
| | 2800 | 1000 | 3 | 1 | 1 | 1332 | 0.813 |
| | 2900 | 1500 | 3 | 0 | 1 | 8663 | 0.796 |
| | **2900** | **1700** | **4** | **2** | **0** | **4098** | **0.904** |

Finally, we observed the effects of different windows and step sizes on false alarms and missed alarms of mixed datasets. According to Figure 4, When window = 2000 and step = 1200, the algorithm successfully detected all drift points of Sine1, and there were no false positives or missed positives. When window = 2000 and step = 1000, the algorithm successfully detected all drift points of Sine2, and there were no false positives or missed positives.

It can be seen that regardless of the type of drift or data feature, the IF-DLDD algorithm can quickly detect the concept drift point. At the same time, we found that different window and step size settings can significantly improve the effect of a certain aspect of measurement, such as low delay, low false alarm, and low missed alarm. For example, the smaller the window size, the lower the missing alarm rate, but the delay is very high, so it is difficult to achieve low delay, low false alarm rate, and low missing alarm rate at the same time. Therefore, in practical application, it is necessary to consider the practical significance to select evaluation indicators.

### 5) PARAMETER SELECTION

The results in Tables 3–6 show that the performance of IF-DLDD is sensitive to the window size and step. Across all artificial datasets, the configurations with the best overall trade-off typically use a window size that is a small fraction of the distance between two neighbouring drifts, and a step that is a fraction of the window length. Empirically, good settings are obtained when the window size is about 5–20% of the expected drift interval, and the step is between roughly 30%




(a) Sine1 (b) Sine2

**FIGURE 4.** The TP, FP and FN values of the different window over Sine1 and Sine2.

**TABLE 7.** Comparison of drift detection algorithms on artificial datasets.

| Dataset | Method | TP | FP | FN | Delay |
|---|---|---|---|---|---|
| SEA | DDM | 3 | 1 | 0 | 6718 |
| | Page-Hinkley | 3 | 3 | 0 | 3364 |
| | HCDTs | 3 | 8 | 0 | 474 |
| | DRPM | 2 | 0 | 1 | 3729 |
| | IF-DLDD | 3 | 0 | 0 | 2591 |
| UG-2C-5D | DDM | 31 | 48 | 68 | 698 |
| | Page-Hinkley | 81 | 2 | 18 | 1182 |
| | HCDTs | 79 | 0 | 20 | 978 |
| | DRPM | 62 | 16 | 37 | 918 |
| | IF-DLDD | 57 | 14 | 42 | 870 |
| Stagger | DDM | 0 | 3 | 3 | NaN |
| | Page-Hinkley | 2 | 8 | 0 | 4998 |
| | HCDTs | 1 | 6 | 1 | 1689 |
| | DRPM | 0 | 0 | 2 | NaN |
| | IF-DLDD | 2 | 2 | 0 | 4577 |
| RotatingHyperplane | DDM | 27 | 19 | 12 | 2375 |
| | Page-Hinkley | 33 | 0 | 6 | 3088 |
| | HCDTs | 39 | 18 | 0 | 1753 |
| | DRPM | 33 | 30 | 6 | 1272 |
| | IF-DLDD | 31 | 9 | 8 | 1870 |
| Sine1 | DDM | 2 | 3 | 2 | 10280 |
| | Page-Hinkley | 4 | 5 | 0 | 998 |
| | HCDTs | 4 | 15 | 0 | 346.5 |
| | DRPM | 4 | 11 | 0 | 4750 |
| | IF-DLDD | 4 | 3 | 0 | 1496 |
| Sine2 | DDM | 1 | 1 | 3 | 15531 |
| | Page-Hinkley | 4 | 5 | 0 | 998 |
| | HCDTs | 4 | 15 | 0 | 127 |
| | DRPM | 4 | 13 | 0 | 5500 |
| | IF-DLDD | 3 | 1 | 1 | 1332 |

and 70% of the window length, reflecting the usual trade-off between stability and responsiveness.

In practice, we recommend the following heuristic. First, based on domain knowledge or a short historical segment, obtain a rough estimate of the minimum distance between drifts. Then initialise the window size within 5–20% of this value and set the step to about one half of the window length. Finally, perform a small local search around this initial configuration to balance false positives, false negatives and delay according to the requirements of the specific application.

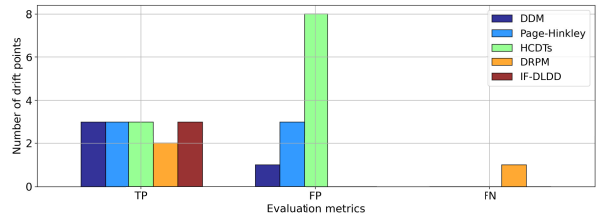
### 6) ALGORITHM COMPARISON

We compare the TP, FP, FN and average delay of four representative concept drift detection methods: (1) DDM, (2) Page-Hinkley, (3) HCDTs and (4) DRPM. The detailed numerical results on the artificial datasets are summarized in Table 7, and the corresponding bar plots are shown in Figures 5–8.

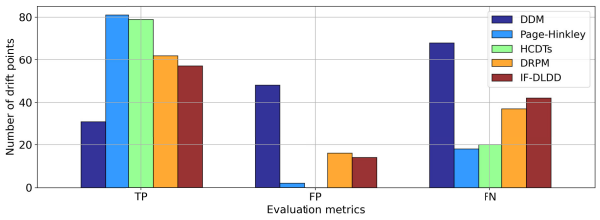
On the abrupt-drift datasets SEA, UG-2C-5D and Stagger, IF-DLDD achieves a favourable balance between detection accuracy and false alarms. For SEA, IF-DLDD detects all three real drifts with no false positives and a smaller delay than DDM and DRPM, while HCDTs attains the smallest delay but at the cost of eight false alarms. On UG-2C-5D, Page-Hinkley and HCDTs obtain slightly larger TP than IF-DLDD, but the difference is mainly due to more aggressive triggering with very small FP; IF-DLDD still detects most drifts and keeps FP at a moderate level. On Stagger, DDM and DRPM fail to detect real drifts, whereas IF-DLDD and Page-Hinkley both detect two of the three real drifts, with IF-DLDD producing fewer false positives than Page-Hinkley.

For the RotatingHyperplane, Sine1 and Sine2, the same pattern is observed. HCDTs consistently achieves the smallest average delay, but this comes with a large number of false alarms. Page-Hinkley also reacts quickly but tends to generate several false positives on noisy streams. DDM and DRPM, which rely on supervised error rates, often miss some drifts or exhibit large delays when the classifier adapts slowly. In contrast, IF-DLDD maintains high recall with substantially fewer false positives than HCDTs and Page-Hinkley, and with delays that are only moderately larger than the fastest method.

Overall, Table 7 and Figures 5–8 show that IF-DLDD offers the most balanced performance across different drift types: it detects the majority of drift points, controls false positives at a moderate level, and incurs only slightly higher delay than the most aggressive method (HCDTs). This behaviour is consistent with the two-layer design of IF-DLDD, where the detection layer quickly responds to changes in the anomaly quantity and the validation

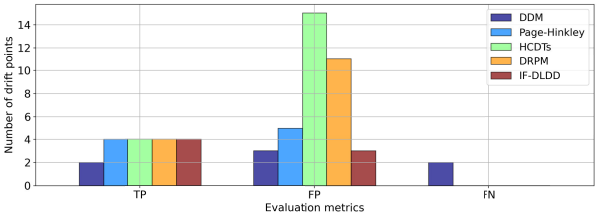



(a) SEA

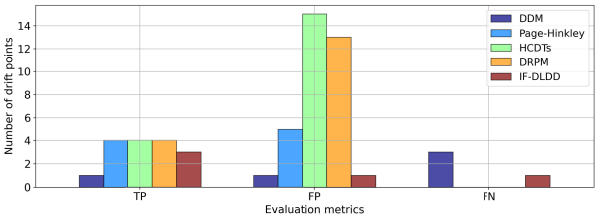



(b) UG-2C-5D

**FIGURE 5.** Evaluation metrics on the SEA,UG-2C-5D dataset.

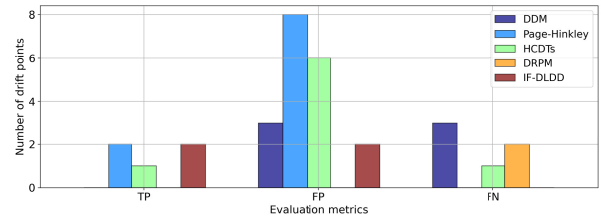



(a) Sine1

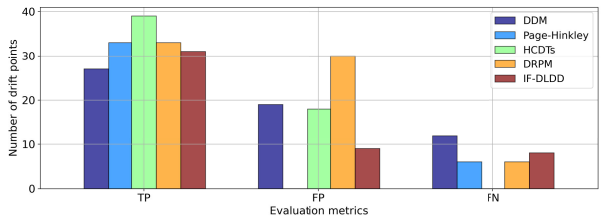



(b) Sine2

**FIGURE 7.** Evaluation metrics on the Sine1,Sine2 dataset.




(a) Stagger




(b) RotatingHyperplane

**FIGURE 6.** Evaluation metrics on the stagger,RotatingHyperplane dataset.

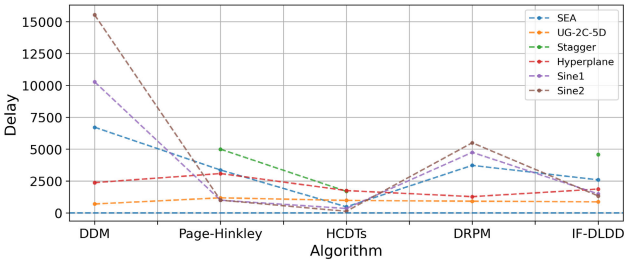



**FIGURE 8.** The delay comparison of various algorithms.

layer filters out noise-induced fluctuations through statistical testing.

In addition to detection performance, we also briefly comment on computational efficiency. In the detection layer, each instance is evaluated by an Isolation Forest with $T$ trees built on subsamples of size $\psi$, so computing one anomaly score costs $O(T \log \psi)$, and updating the anomaly count in the sliding window is $O(1)$. The validation layer is triggered only when a potential drift window is flagged, and the t-test on a window of length $n_w$ has linear cost $O(n_w)$, so its amortized cost per instance is small. Overall, the per-instance complexity of IF-DLDD is therefore dominated by the $O(T \log \psi)$ Isolation Forest evaluation, which remains efficient in streaming scenarios. Compared with DDM and Page-Hinkley, which achieve $O(1)$ time per instance at the expense of requiring labelled data, and with more complex layered methods such as HCDTs and DRPM, IF-DLDD provides a good compromise between computational efficiency and robust, label-free drift detection.

### C. INTRUSION DETECTION EVALUATION DATASET

With the rapid development of the IoT, the number of IoT devices and systems has increased dramatically, and cyber threats and attacks are constantly changing. Network attacks are characterized by complex methods, variable attack duration and high harm, making them a threat to network security. It is of great significance to identify network attacks accurately from network data stream for network space security. This section focuses on how concept drift affects network attack detection in the IoT scenario.

#### 1) BASIC DATA INFORMATION

To accomplish our purpose, we apply for the dataset CIC-IDS2017, which is available at the website https://www.unb.ca/cic/datasets/ids-2017.html. CIC-IDS2017 is popular in network intrusion detection problems [43]. It contains benign and up-to-date common attacks. The data capturing period begins at 9 am on Monday, July 3, 2017, and ends at 5pm on Friday, July 7, 2017. Monday only contains benign traffic, whereas there are types of attacks from Tuesday to Friday.

**TABLE 8.** Data distribution of CIC-IDS2017 dataset.

| Day | Sample | Feature | Type of traffic | Classified samples |
|---|---|---|---|---|
| Tuesday | 445909 | 84 | Benign | 432074 |
| | | | SSH-Patator | 5897 |
| | | | FTP-Patator | 7938 |
| Wednesday | 692703 | 84 | Benign | 440031 |
| | | | DoS Hulk | 231073 |
| | | | DoS GoldenEye | 10293 |
| | | | DoS Slowloris | 5796 |
| | | | DoS Slowhttptest | 5499 |
| | | | Heartbleed | 11 |
| Thursday | 458968 | 84 | Benign | 456752 |
| | | | Web Attack-Brute Force | 1507 |
| | | | Web Attack-Sql Injection | 21 |
| | | | Web Attack-XSS | 652 |
| | | | Infiltration | 36 |
| Friday | 703245 | 84 | Benign | 414322 |
| | | | Bot | 1966 |
| | | | Portscan | 158930 |
| | | | DDoS | 128027 |
| Total samples | | | | 2830743 |

**TABLE 9.** CIC-IDS2017 feature selection.

| No. | Feature | Gain | No. | Feature | Gain |
|---|---|---|---|---|---|
| 1 | Flow Bytes/s | 1.612 | 6 | Average Packet Size | 1.562 |
| 2 | Flow Packets/s | 1.585 | 7 | Bwd Packets/s | 1.530 |
| 3 | Fwd Packets/s | 1.578 | 8 | Packet Length Mean | 1.526 |
| 4 | Flow IAT Mean | 1.572 | 9 | Flow IAT Max | 1.522 |
| 5 | Flow Duration | 1.569 | 10 | Packet Length Std | 1.488 |

CIC-IDS2017 [44] covers diverse attacks that can effectively reflect the current trend of network attack intrusion. Table 8 lists its detailed data distribution.

### 2) FEATURE SELECTION

There is redundancy in the original data, i.e., the feature Fwd Header Length is duplicated. In this paper, we first remove one of the columns. Moreover, to improve the accuracy of traffic anomaly detection and reduce its execution time, this study first analyzes the significant characteristics of the network traffic. Information gain is the most commonly used feature selection technique in intrusion detection system research [45], [46] In this paper, 56661 samples were randomly selected as a random subset of the CIC-IDS2017 dataset. We deleted seven non-numerical features and descending sorted the remaining 76 features according to the information gain. Finally, we select the top 10 features shown in Table 9 to evaluate the proposed algorithm.

### 3) DESIGN CONCEPT

To investigate the behaviour of IF-DLDD under different types of distributional shift in network attack detection [47], [48], [49], [50], [51], [52], we construct a synthetic data stream from CIC-IDS2017 by explicitly designing five concepts C1-C5 with predefined class-level compositions. The goal is to emulate five representative intrusion scenarios: C1 contains purely benign traffic, C2 is roughly balanced between benign flows and Patator attacks, C3 represents the sudden emergence of a large-scale DoS attack, C4 corresponds to mostly benign traffic with only a small number of web attacks during specific periods and C5 represents a severe attack-dominated regime with a high proportion of Bot, Portscan and DDoS traffic. Each concept consists of exactly 20,000 flow records, and the desired number of samples for each traffic type within each concept is specified in Table 10.

In constructing these concepts, we do not preserve the original attack-to-benign ratios of each day in CIC-IDS2017. Instead, for each traffic type and each concept, we first determine the target sample size according to Table 10 and then perform simple random sampling without replacement from the corresponding subset of CIC-IDS2017 where that traffic type appears. The sampled subsets for all traffic types are concatenated and randomly shuffled to form the final 20000 sample concept. The complete data stream is obtained by concatenating C1-C5 in order, resulting in 100000 samples with drift points at indices 20001, 40001, 60001 and 80001.

**TABLE 10.** The designed concepts of CIC-IDS2017.

| Concept | Sample | Feature | Type of traffic | Classified samples |
|---|---|---|---|---|
| C1 | 20000 | 10 | Benign | 20000 |
| C2 | 20000 | 10 | Benign | 8000 |
| | | | SSH-Patator | 5000 |
| | | | FTP-Patator | 7000 |
| C3 | 20000 | 10 | Benign | 5000 |
| | | | DoS Hulk | 13687 |
| | | | DoS GoldenEye | 598 |
| | | | DoS Slowloris | 346 |
| | | | DoS Slowhttptest | 359 |
| | | | Heartbleed | 10 |
| C4 | 20000 | 10 | Benign | 17970 |
| | | | Web Attack-Brute Force | 1384 |
| | | | Web Attack-Sql Injection | 18 |
| | | | Web Attack-XSS | 598 |
| | | | Infiltration | 30 |
| C5 | 20000 | 10 | Benign | 2000 |
| | | | Bot | 1000 |
| | | | Portscan | 7000 |
| | | | DDoS | 10000 |

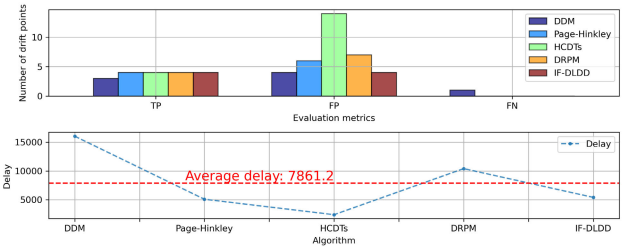



**FIGURE 9.** Compare the detection performance and delay of CIC-IDS2017 dataset on various algorithms.

**TABLE 11.** Results on drift point detection on the CIC-IDS2017 dataset by IF-DLDD.

| **Evaluation metrics** | | | | | | | | | | **TP** | **FP** | **FN** | **Delay** | $F_1$ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Real drifts | | | **20001** | | **40001** | | **60001** | | **80001** | 4 | 0 | 0 | 0 | 1.000 |
| win=1500, step=800 | 4801 | 17601 | 20801 | 35201 | 57601 | | 64001 | 75201 | 80001 | 4 | 4 | 0 | 5600 | 0.850 |
| win=1500, step=1000 | 10001 | | 36001 | | 42001 | 56001 | 66001 | | 92001 | 4 | 2 | 0 | 9000 | 0.894 |
| win=1500, step=1300 | | | 36401 | | 57201 | | 72801 | | - | 3 | 0 | 1 | 15466 | 0.789 |
| win=1600, step=1000 | 10001 | | 36001 | | - | | 60001 | 66001 | 92001 | 3 | 2 | 1 | 9333 | 0.754 |
| win=1600, step=1300 | | | 36401 | | 57201 | | 72801 | | 91001 | 4 | 0 | 0 | 14350 | 0.935 |
| win=1800, step=1000 | 4001 | 8001 | 20001 | 36001 | - | | 60001 | 72001 | 92001 | 3 | 4 | 1 | 4000 | 0.718 |
| win=2000, step=1700 | | | 34001 | | 54401 | | 61201 | 74801 | 81601 | 4 | 1 | 0 | 7800 | 0.930 |

### D. RESULTS ON CIC-IDS2017 DATASETS

In the case of network intrusion, the statistical characteristics of network traffic constantly change because the attacker will alter the mode and frequency on purpose. It requires that the algorithm detect the occurrence of attack drift in time and maintain a high detection and recognition rate.

According to Table 11, the real drift point in CIC-IDS2017 is 20001, 40001, 60001, 80001. Window sizes range from 1,500 to 2000, and sliding steps from 800 to 1700. It shows the drift detection results under different windows and step sizes. The bold numbers are the correctly detected locations of the drift points, and some of them are delayed. The normal number is the location of the error-detected drift point. According to the results in the table, the setting with window $=$ 1600 and step $=$ 1300 achieves the best overall performance, withTP=4, FP $= 0$, FN $= 0$, Delay $=$ 14350 and $F_1 = 0.935$. This setting is consistent with the construction of the stream, each concept segment contains 20000 instances, so a window of 1600 is only about 8% of the concept length—large enough to smooth short-term fluctuations, yet small enough for the anomaly quantity to respond promptly when the attack–benign composition changes. A step of 1300 leaves a moderate overlap of 300 instances between neighbouring windows, ensuring that at least one window straddles each real drift point without generating an excessive number of highly redundant candidate windows. In contrast, smaller steps produce many overlapping windows with little improvement in $F_1$, while larger windows or steps reduce the number of windows per concept and tend to increase delay or cause occasional missed drifts.

Finally, we compared the performance of CIC-IDS2017 dataset on five drift detection algorithms. As shown in Figure 9, IF-DLDD algorithm detected all drift points, with the least false positives and no missed positives. The average delay of the five drift detection algorithms is 7861.2, among which the delay of HCDTs is the lowest, but there are a lot of false positives, and the delay of IF-DLDD algorithm is lower than the average.

## V. CONCLUSION

Concept drift in data streams poses a critical challenge to maintaining reliable model performance, especially in real-world applications where labeled data is scarce and costly. To address this issue, we proposed IF-DLDD, a double-layer drift detection framework based on isolation forest. By integrating anomaly quantity analysis with distributional verification, IF-DLDD effectively localizes drift points while reducing false positives. Experimental results on both synthetic datasets and the CIC-IDS2017 intrusion detection subset demonstrate that IF-DLDD can accurately and promptly detect diverse types of concept drift with low delay and strong robustness to noise, confirming its practical applicability in dynamic data stream environments.

Despite its effectiveness, IF-DLDD still requires manually defined parameters such as window and step size, which may limit adaptability in complex scenarios. Future work will focus on designing adaptive parameter selection mechanisms, incorporating more advanced statistical tools, and exploring hybrid approaches that combine unsupervised drift detection with lightweight online learning. We also plan to extend the framework to large-scale, heterogeneous IoT environments and evaluate its performance under real-time operational constraints. These directions aim to enhance the scalability, flexibility, and practicality of IF-DLDD for detecting and mitigating concept drift in evolving cyber-attack and IoT data stream scenarios.

## DECLARATION OF COMPETING INTEREST

All authors certify that they have no affliations with or involvement in any organization or entity with any financial interest or non-financial interest in the subject matter or materials discussed in this manuscript.

## REFERENCES


[1] R. Mishra and A. Mishra, "Current research on Internet of Things (IoT) security protocols: A survey," *Comput. Secur.*, vol. 151, Apr. 2025, Art. no. 104310.

[2] E. Benkhelifa, T. Welsh, and W. Hamouda, "A critical review of practices and challenges in intrusion detection systems for IoT: Toward universal and resilient systems," , vol. 20, no. 4, pp. 3496–3509, 4th Quart., 2018.

[3] M. M. Rahman, S. A. Shakil, and M. R. Mustakim, "A survey on intrusion detection system in IoT networks," *Cyber Secur. Appl.*, vol. 3, Dec. 2025, Art. no. 100082.

[4] K. Wang, J. Lu, A. Liu, Y. Song, L. Xiong, and G. Zhang, "Elastic gradient boosting decision tree with adaptive iterations for concept drift adaptation," *Neurocomputing*, vol. 491, pp. 288–304, Jun. 2022.

[5] J. C. Schlimmer and R. H. Granger, "Incremental learning from noisy data," *Mach. Learn.*, vol. 1, no. 3, pp. 317–354, Sep. 1986.

[6] F. Bayram, B. S. Ahmed, and A. Kassler, "From concept drift to model degradation: An overview on performance-aware drift detectors," *Knowl.-Based Syst.*, vol. 245, Jun. 2022, Art. no. 108632.

[7] D. Shang, G. Zhang, and J. Lu, "Novelty-aware concept drift detection for neural networks," *Neurocomputing*, vol. 617, Feb. 2025, Art. no. 128933.

[8] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection and concept-drift adaptation with delayed supervised information," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2015, pp. 1–8.

[9] C. Yanyang and J. Zhu, "ACLEGR: Adaptive continual learning with efficient generative replay for financial fraud detection under concept drift," *Authorea Preprints*, 2025.

[10] A. Somasundaram and S. Reddy, "Parallel and incremental credit card fraud detection model to handle concept drift and data imbalance," *Neural Comput. Appl.*, vol. 31, no. S1, pp. 3–14, Jan. 2019.

[11] M. Jain, G. Kaur, and V. Saxena, "A K-means clustering and SVM based hybrid concept drift detection technique for network anomaly detection," *Expert Syst. Appl.*, vol. 193, May 2022, Art. no. 116510.

[12] D. Mulimani, S. G. Totad, P. Patil, and S. V. Seeri, "Adaptive ensemble learning with concept drift detection for intrusion detection," in *Data Engineering and Intelligent Computing*, 2021, pp. 331–339.

[13] W. S. Jones and D. J. Farrow, "One-class support vector machines for detecting population drift in deployed machine learning medical diagnostics," *Sci. Rep.*, vol. 15, no. 1, p. 12157, Apr. 2025.

[14] R. A. Rios, T. N. Rios, R. Melo, E. S. de Santana, T. M. S. Carneiro, and A. D' Oliveira, "Applying concept drift to understand hepatitis evolution in Brazil," *Cybern. Syst.*, vol. 51, no. 6, pp. 631–645, Aug. 2020.

[15] L. L. Minku, A. P. White, and X. Yao, "The impact of diversity on online ensemble learning in the presence of concept drift," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 5, pp. 730–742, May 2010.

[16] H. Gâlmeanu and R. Andonie, "Weighted incremental–decremental support vector machines for concept drift with shifting window," *Neural Netw.*, vol. 152, pp. 528–541, Aug. 2022.

[17] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 12, pp. 2346–2363, Dec. 2019.

[18] X. Wu, P. Li, and X. Hu, "Learning from concept drifting data streams with unlabeled data," *Neurocomputing*, vol. 92, pp. 145–155, Sep. 2012.

[19] E. Lughofer, E. Weigl, W. Heidl, C. Eitzinger, and T. Radauer, "Recognizing input space and target concept drifts in data streams with scarcely labeled and unlabelled instances," *Inf. Sci.*, vols. 355–356, pp. 127–151, Aug. 2016.

[20] C. H. Tan, V. C. S. Lee, and M. Salehi, "Information resources estimation for accurate distribution-based concept drift detection," *Inf. Process. Manage.*, vol. 59, no. 3, May 2022, Art. no. 102911.

[21] J. Gama, P. Medas, G. Castillo, and P. P. Rodrigues, "Learning with drift detection," in *Proc. Brazilian Symp. Artif. Intell.*, 2004, pp. 286–295.

[22] K. Nishida and K. Yamauchi, "Detecting concept drift using statistical testing," in *Proc. Int. Conf. Discovery Sci.*, 2007, pp. 264–269.

[23] L. Bu, C. Alippi, and D. Zhao, "A pdf-free change detection test based on density difference estimation," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 2, pp. 324–334, Feb. 2018.

[24] H. Qiao, B. Novikov, and J. O. Blech, "Concept drift analysis by dynamic residual projection for effectively detecting botnet cyber-attacks in IoT scenarios," *IEEE Trans. Ind. Informat.*, vol. 18, no. 6, pp. 3692–3701, Jun. 2022.

[25] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM Int. Conf. Data Mining*, Apr. 2007, pp. 443–448.

[26] X. Wang, Q. Kang, J. An, and M. Zhou, "Drifted Twitter spam classification using multiscale detection test on K-L divergence," *IEEE Access*, vol. 7, pp. 108384–108394, 2019.

[27] C. Alippi, G. Boracchi, and M. Roveri, "Hierarchical change-detection tests," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 2, pp. 246–258, Feb. 2017.

[28] Y. Zhang, G. Chu, P. Li, X. Hu, and X. Wu, "Three-layer concept drifting detection in text data streams," *Neurocomputing*, vol. 260, pp. 393–403, Oct. 2017.

[29] S. Yu, X. Wang, and J. C. Principe, "Request-and-reverify: Hierarchical hypothesis testing for concept drift detection with expensive labels," 2018, *arXiv:1806.10131*.

[30] X. Wang, Q. Kang, M. Zhou, L. Pan, and A. Abusorrah, "Multiscale drift detection test to enable fast learning in nonstationary environments," *IEEE Trans. Cybern.*, vol. 51, no. 7, pp. 3483–3495, Jul. 2021.

[31] M. Baena-García, J. del Campo-Ávila, R. Fidalgo, A. Bifet, R. Gavalda, and R. Morales-Bueno, "Early drift detection method," in *Proc. 4th Int. Workshop Knowl. Discovery Data Streams*, 2006, pp. 77–86.

[32] A. Liu, G. Zhang, and J. Lu, "Fuzzy time windowing for gradual concept drift adaptation," in *Proc. IEEE Int. Conf. Fuzzy Syst. (FUZZ-IEEE)*, Jul. 2017, pp. 1–6.

[33] S. Greco, B. Vacchetti, D. Apiletti, and T. Cerquitelli, "Unsupervised concept drift detection from deep learning representations in real-time," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 10, pp. 6232–6245, Oct. 2025.

[34] S. Yu, Z. Abraham, H. Wang, M. Shah, Y. Wei, and J. C. Príncipe, "Concept drift detection and adaptation with hierarchical hypothesis testing," *J. Franklin Inst.*, vol. 356, no. 5, pp. 3187–3215, Mar. 2019.

[35] H. Wang and Z. Abraham, "Concept drift detection for streaming data," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2015, pp. 1–9.

[36] A. Augello, A. De Paola, and G. Lo Re, "Hybrid multilevel detection of mobile devices malware under concept drift," *J. Netw. Syst. Manage.*, vol. 33, no. 2, p. 36, Apr. 2025.

[37] L. Zhang, J. Zhao, and W. Li, "Online and unsupervised anomaly detection for streaming data using an array of sliding windows and PDDs," *IEEE Trans. Cybern.*, vol. 51, no. 4, pp. 2284–2289, Apr. 2021.

[38] T. Zoppi, A. Ceccarelli, and A. Bondavalli, "MADneSs: A multi-layer anomaly detection framework for complex dynamic systems," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 2, pp. 796–809, Mar. 2021.

[39] Y. Wu, L. Liu, Y. Yu, G. Chen, and J. Hu, "Online ensemble learning-based anomaly detection for IoT systems," *Appl. Soft Comput.*, vol. 173, Apr. 2025, Art. no. 112931.

[40] Z. Ding and M. Fei, "An anomaly detection approach based on isolation forest algorithm for streaming data using sliding window," *IFAC Proc. Volumes*, vol. 46, no. 20, pp. 12–17, 2013.

[41] F. T. Liu, K. M. Ting, and Z. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining*, 2008, pp. 413–422.

[42] W. N. Street and Y. Kim, "A streaming ensemble algorithm (SEA) for large-scale classification," in *Proc. 7th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2001, pp. 377–382.

[43] I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, 2018, pp. 108–116.

[44] Kurniabudi, D. Stiawan, Darmawijoyo, M. Y. Bin Idris, A. M. Bamhdi, and R. Budiarto, "CICIDS-2017 dataset feature analysis with information gain for anomaly detection," *IEEE Access*, vol. 8, pp. 132911–132921, 2020.

[45] M. Kudo and J. Sklansky, "Comparison of algorithms that select features for pattern classifiers," *Pattern Recognit.*, vol. 33, no. 1, pp. 25–41, Jan. 2000.

[46] S. Ramírez-Gallego, B. Krawczyk, S. García, M. Woźniak, and F. Herrera, "A survey on data preprocessing for data stream mining: Current status and future directions," *Neurocomputing*, vol. 239, pp. 39–57, May 2017.

[47] H. Qiao, J. O. Blech, and H. Chen, "A machine learning based intrusion detection approach for industrial networks," in *Proc. IEEE Int. Conf. Ind. Technol. (ICIT)*, Feb. 2020, pp. 265–270.

[48] M. A. Ferrag, L. Maglaras, S. Moschoyiannis, and H. Janicke, "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study," *J. Inf. Secur. Appl.*, vol. 50, Feb. 2020, Art. no. 102419.

[49] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, pp. 1–58, 2009.

[50] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surv.*, vol. 46, no. 4, pp. 1–37, Apr. 2014.
[51] D. Liu, H. Zhen, D. Kong, X. Chen, L. Zhang, M. Yuan, and H. Wang, "Sensors anomaly detection of industrial Internet of Things based on isolated forest algorithm and data compression," *Sci. Program.*, vol. 2021, pp. 1–9, Jan. 2021.
[52] A. Kuppa and N.-A. Le-Khac, "Learn to adapt: Robust drift detection in security domain," *Comput. Electr. Eng.*, vol. 102, Sep. 2022, Art. no. 108239.



**RENJIE CHU** received the B.S. degree in applied statistics from Changchun University, Changchun, China, in 2020, and the M.S. degree in applied statistics from Guilin University of Technology, Guilin, China, in 2022. She is currently pursuing the Ph.D. degree with the School of Data Sciences, Zhejiang University of Finance and Economics, Hangzhou, China. From July 2022 to July 2024, she was with Guilin University. Her research interests include data streams, machine learning, and change point detection.



**MENGZHEN LUO** received the B.S. degree in computer science and technology and the M.S. degree in computer software and theory from Guangxi Normal University, Guilin, China, in 2006 and 2011, respectively. She is currently an Associate Professor with the School of Information Engineering, Guilin University, Guilin. Her research interests include artificial intelligence, big data, and computer software and theory.



**LEI YANG** received the B.S. and M.S. degrees from Guangxi Normal University, Guilin, China, in 2005 and 2014, respectively. She is currently a Senior Engineer and a Faculty Member with Guilin University, Guilin. Her research interests include big data applications and intelligent recommendation algorithms.



**JIANMING XIAO** received the M.S. degree in signal and information processing from Guilin University of Electronic Technology, in 2006. He is currently a Faculty Member with the School of Information Engineering, Guilin University. His research interests include navigation, positioning, and location-based services.



**YUANYUAN LIAO** received the B.S. degree in computer science and technology from Guangxi Normal University, Guilin, China, in 2003. She is currently an Associate Professor with the School of Information Engineering, Guilin University, Guilin. Her research interests include cloud computing technology, database technology, and modern educational technology.